

CS272 Programming Assignment 2

Virus Spreading through a Social Network with SARSA(λ)

Author: Venkat Anoop Karlapudi

1 Introduction

This project is about building a small virus-spread simulation and training an agent to use it. A social network is represented as a graph, where each person has connections to other people and a different chance of becoming infected (infection probability is conceptually the opposite of immunity against the virus). A SARSA(λ) agent with eligibility traces is then trained to choose infection targets and spread the virus as much as possible. The agent is tested with different λ values and random seeds, and its results are compared with a random-agent baseline.

Not all people in a social network are connected to each other, so the agent first has to discover the structure of the graph itself through trial and error, and then figure out a way to avoid highly immune individuals who can cause the transmission to stop.

A new seed value generates a completely new graph with different connections and immunities, so the environment is very stochastic.

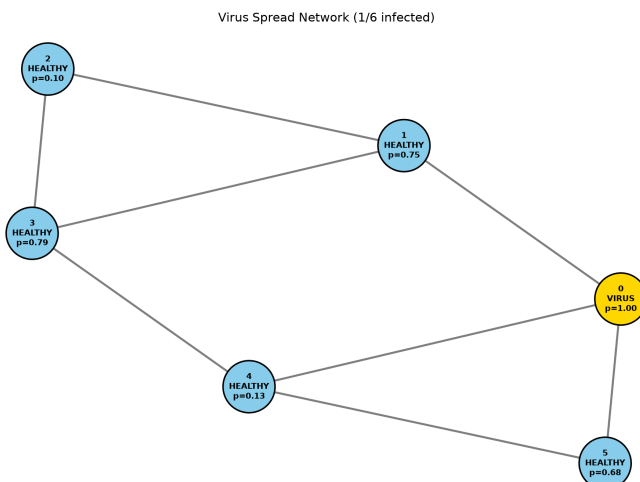
2 Repository

The code for this project is available here:

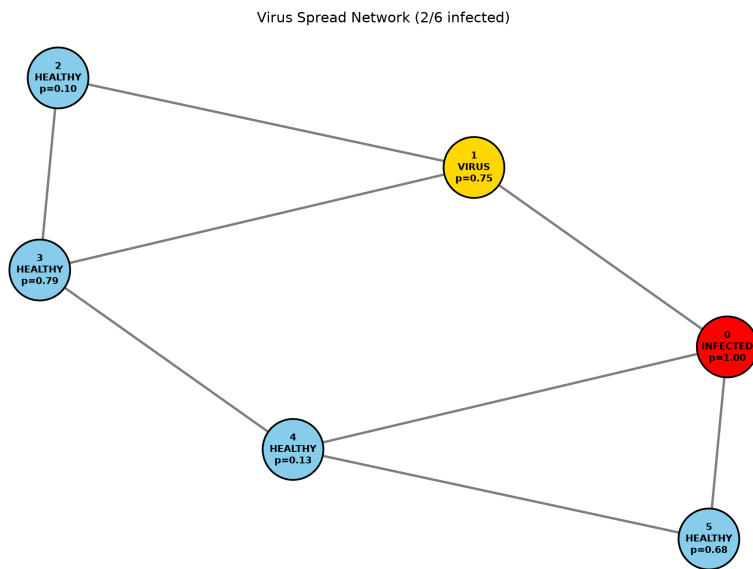
<https://github.com/Anoop-Grunt/cs-272-pa2>

3 Environment and States

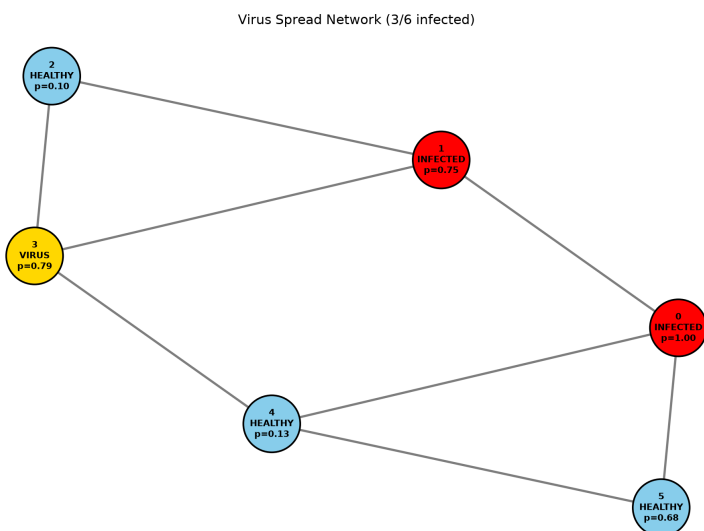
At step 0/ initially, everyone is healthy except for person 0. The virus starts at person 0 (patient zero is always infected), and tries to spread through the network.



Then it successfully transmits to person 1, who has a direct edge from 0, and a low immunity (0.75 chance of infection)



Then after 2 failed attempts at transmission, it successfully infects person 3



This would leave it with one more attempt left.

The episode terminates if failed transmissions equal or exceed `max_failures`, or the `goal_count` amount of people have been infected, which in this case is equal to `n_people` (so everyone has to get infected).

4 Sample Greedy Episode

Episodic returns are heavily seed dependent, and greedy policies aren't automatically the best choice here, because repeatedly failing to transmit to the same person can cause premature termination of the episode like shown below.

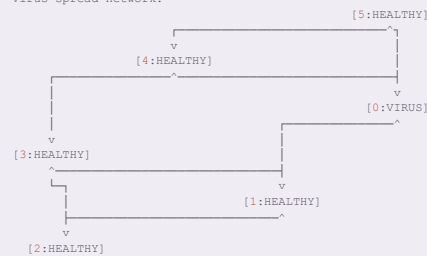
Best run data:

```
Step 1: current=0, infected=[0], action=5, reward=1.00
Step 2: current=5, infected=[0, 5], action=0, reward=-0.30
Step 3: current=0, infected=[0, 5], action=1, reward=1.00
Step 4: current=1, infected=[0, 1, 5], action=3, reward=1.00
Step 5: current=3, infected=[0, 1, 3, 5], action=4, reward=-0.75
Step 6: current=3, infected=[0, 1, 3, 5], action=4, reward=-0.75
Step 7: current=3, infected=[0, 1, 3, 5], action=4, reward=-0.75
Reached terminal: True
Undiscounted return: 0.4500000000000002
Discounted return: 0.5135993430242503
```

ASCII RENDERED EPISODE

The following is one greedy evaluation episode after training. The environment was reset with seed 0, and the episode was rendered using the ANSI renderer.

fully greedy policy episode in ASCII : DIFFERENT SEED - SO DIFFERNT GRAPH FROM THE BEST RUN EPISODE
Virus spread network:



Current person: 0
Infected count: 1/6
Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

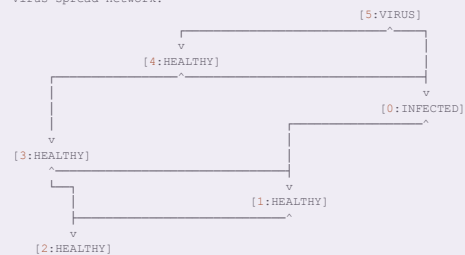
Step 1

Observation: 6

Action: 5

Reward: 1.0

Virus spread network:



Current person: 5
Infected count: 2/6
Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

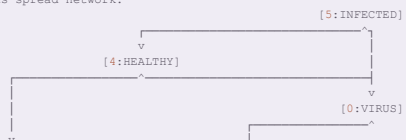
Step 2

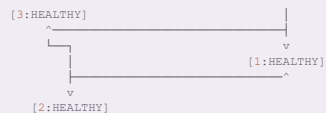
Observation: 203

Action: 0

Reward: -0.3

Virus spread network:

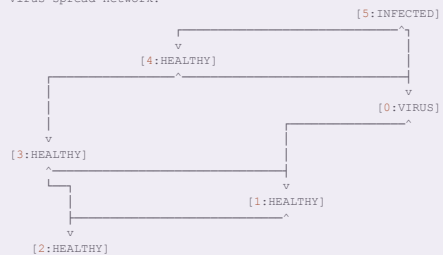




Current person: 0
 Infected count: 2/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

Step 3
 Observation: 198
 Action: 1
 Reward: -0.75

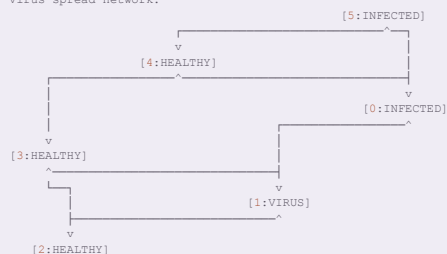
Virus spread network:



Current person: 0
 Infected count: 2/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

Step 4
 Observation: 198
 Action: 1
 Reward: 1.0

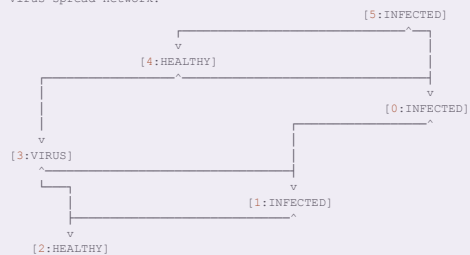
Virus spread network:



Current person: 1
 Infected count: 3/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

Step 5
 Observation: 211
 Action: 3
 Reward: 1.0

Virus spread network:

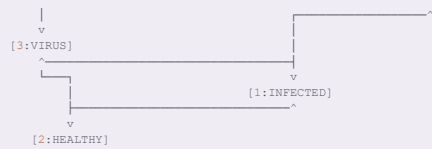


Current person: 3
 Infected count: 4/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

Step 6
 Observation: 261
 Action: 4
 Reward: -0.75

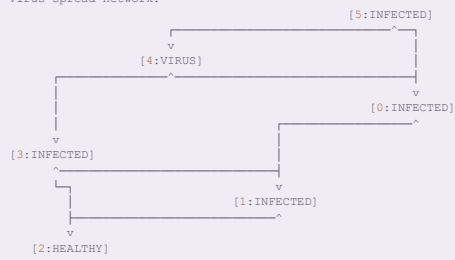
Virus spread network:





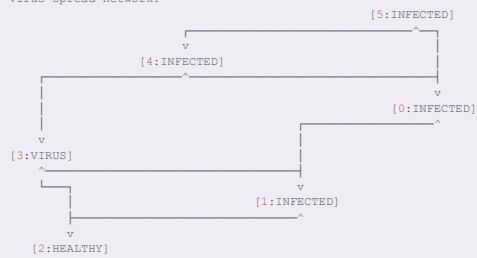
Current person: 3
 Infected count: 4/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

Step 7
 Observation: 261
 Action: 4
 Reward: 1.0
 Virus spread network:



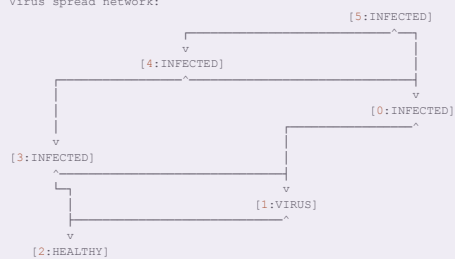
Current person: 4
 Infected count: 5/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

Step 8
 Observation: 358
 Action: 3
 Reward: -0.3
 Virus spread network:



Current person: 3
 Infected count: 5/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

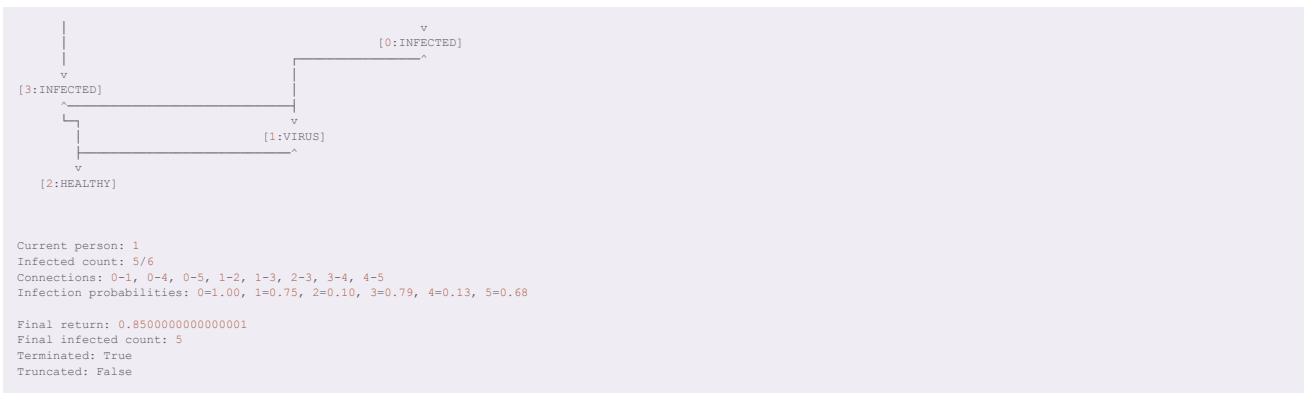
Step 9
 Observation: 357
 Action: 1
 Reward: -0.3
 Virus spread network:



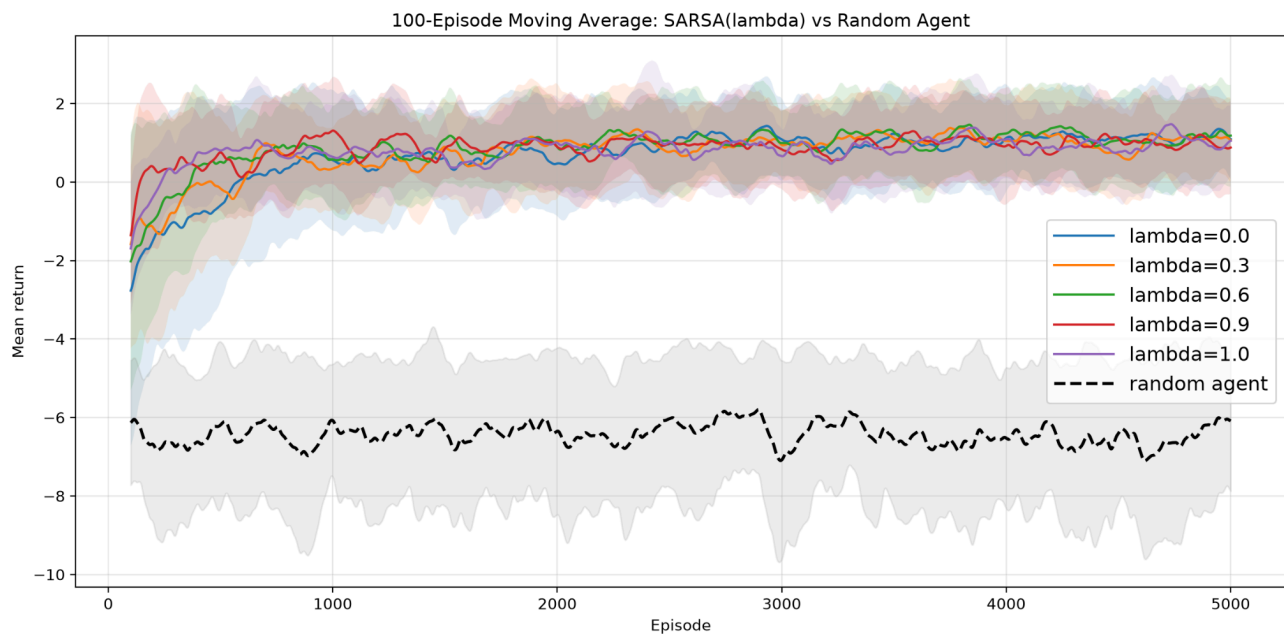
Current person: 1
 Infected count: 5/6
 Connections: 0-1, 0-4, 0-5, 1-2, 1-3, 2-3, 3-4, 4-5
 Infection probabilities: 0=1.00, 1=0.75, 2=0.10, 3=0.79, 4=0.13, 5=0.68

Step 10
 Observation: 355
 Action: 2
 Reward: -0.75
 Virus spread network:





5 Lambda Sweep Plot



A 100-point moving average of returns, for each λ value as indicated by the color codes in the legend. The lines indicate the actual mean values across all the seeds, and the fill-colors around the lines (the transparent backgrounds) indicate the actual spread of the returns across all the seed values used in the experiment.

As we can see, the spread is wider during early training, but becomes considerably narrower towards the end. However, there spread never really disappears, so there's always some variation across seeds.

The dotted line, and the spread around it indicate the random baseline returns. As we can see, the random agent performs significantly worse than the SARSA λ agent, and has much more spread across the seed values.

5.1 Results Table

The target is a return threshold of 1.0

lambda results

lambda	first return threshold	mean final return	mean final infected
0.0	1795	1.071	4.176
0.3	697	1.041	4.198
0.6	776	1.184	4.106
0.9	851	0.894	3.878
1.0	650	1.060	3.942

Target return threshold: 1.0

Moving-average window: 100

As we can see, the threshold is reached anywhere between 675-1795 episodes depending on the λ value we choose.

The big takeaway here is that a λ of 0.6 leads to the most optimal returns, but higher values don't tend to improve consistently, in fact they suffer, possibly because of noise propagating backwards.

At the end of the 5000 episode training, we are able to infect an average of 4.1/6 people in the network before terminating.

5.2 Why Lambda Changes the Results

In this environment, λ controls how far the learning signal is carried backward through the episode. With $\lambda=0$, only the most recent state-action pair receives credit. Larger λ values also give credit to earlier decisions that helped reach a successful infection. This matters because the agent often needs to make several connected moves before reaching a healthy person.

A moderate value, such as $\lambda=0.6$, worked best in these experiments. It allowed successful infection rewards to reach earlier decisions without spreading the update too far. This is why $\lambda=0.6$ reached the return threshold fastest and had the highest mean final return. The larger value $\lambda=0.9$ likely did not perform as well because the environment is stochastic. Failed transmissions, invalid moves, and revisiting infected people can produce noisy rewards, and a large eligibility trace spreads those noisy updates across more previous actions. The results therefore do not improve consistently as λ increases.

The final infected counts are fairly similar for all λ values, staying close to four people. This suggests that λ mainly changed how quickly the agent learned useful decisions, rather than changing the best outcome the environment allowed. In this environment, moderate eligibility traces were more useful than either one-step SARSA or very long traces.

5.2.1 Testing how lambda affects larger graphs

So, one possibility is that the graph we chose here is small enough that the true power of the λ trace isn't put to use. We can verify this by making a larger graph of let's say 10 people, where the discrete state space would be much larger than 500 (so it's not part of the original experiment). In such a large graph, where 5000 episodes aren't sufficient, we might really see the larger lambda values have a greater impact.

This was verified with a 10 person graph, with results below.

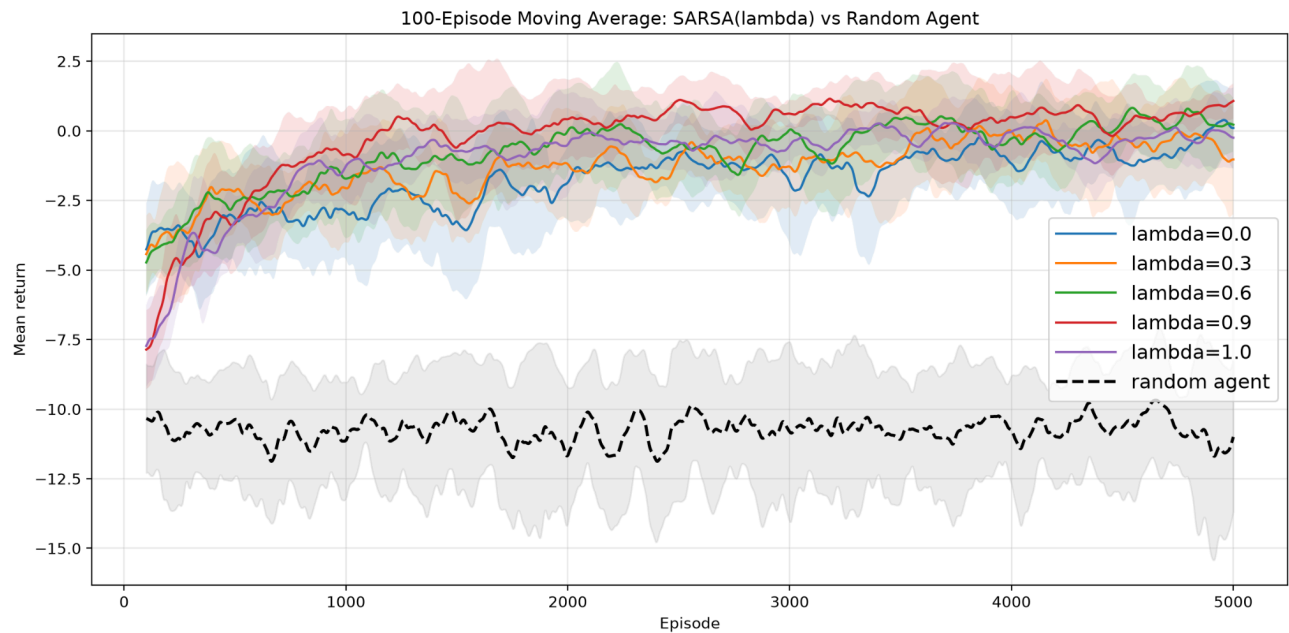
lambda results

lambda	first return threshold	mean final return	mean final infected
0.0	never	0.116	5.326
0.3	never	-1.045	5.034
0.6	never	0.208	5.076
0.9	2478	1.101	5.348
1.0	never	-0.256	3.812

Target return threshold: 1.0

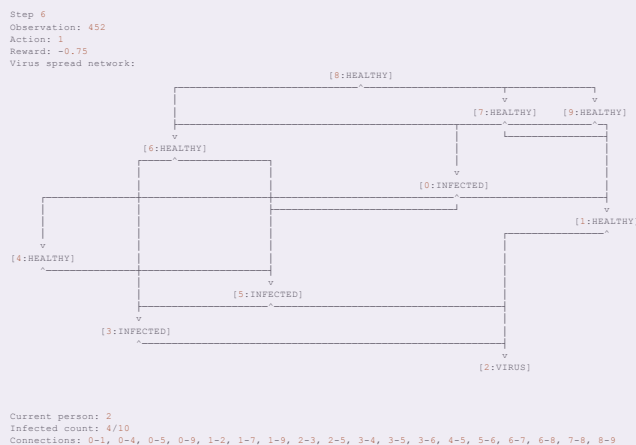
Moving-average window: 100

The same agent trained over a much larger graph → takes much longer to reach the threshold, and only reaches it with a VERY large lambda value of 0.9.



Lambda sweep for the larger graph also shows that larger lambda values become more useful when the state trajectories are longer. Again, this experiment has a much larger state space, and is just being used to test the hypothesis earlier about the power of lambda traces being brought out in longer trajectories.

Example of a terminating state in this graph:



```
Infection probabilities: 0=1.00, 1=0.49, 2=0.81, 3=0.85, 4=0.39, 5=0.56, 6=0.36, 7=0.58, 8=0.37, 9=0.41  
Final return: 0.75  
Final infected count: 4  
Terminated: True  
Truncated: False
```

6 Reproducibility Params

```
reproducibility parameters  
-----  
environment           : cs272/MyEnv-v0  
people                : 6  
episodes per run      : 5000  
lambda values         : [0.0, 0.3, 0.6, 0.9, 1.0]  
seeds                 : [0, 1, 2, 3, 4]  
gamma                 : 0.99  
alpha                 : 0.05  
epsilon               : 0.15  
initial q value       : 1.0  
trace type            : accumulating  
moving-average window : 100  
target return         : 1.0
```